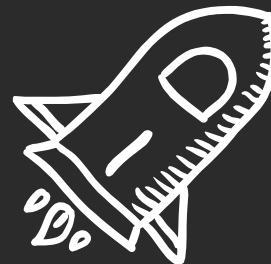




Semana 11 ¡Bienvenidos!



Temas de hoy

1

ORM - Object Relational
Mapper o Mapeador
Objeto-Relacional

2

Relaciones entre
modelos con Django

3

Operaciones CRUD con
el ORM de Django

1

ORM

En la clase anterior vimos modelos en Django, y te mencionamos cómo trabajan, pero ahora vamos a profundizar un poco más.

ORM (Object-Relational Mapping) es una técnica que permite interactuar con bases de datos relacionales utilizando el paradigma orientado a objetos de un lenguaje de programación. En lugar de escribir consultas SQL directamente, los desarrolladores pueden manipular bases de datos utilizando objetos y métodos del lenguaje.

ORM - Object Relational Mapper o Mapeador Objeto-Relacional

Ventajas del ORM

El ORM (Object-Relational Mapper) de Django ofrece numerosas ventajas que lo convierten en una herramienta esencial para cualquier desarrollador Django:

→ **Abstracción de la base de datos:**

- *Independencia de la base de datos:* Django ORM te permite cambiar de un sistema de gestión de bases de datos a otro (por ejemplo, de SQLite a PostgreSQL) con mínimos cambios en tu código.
- *Simplificación de consultas:* En lugar de escribir complejas consultas SQL, puedes utilizar la sintaxis de Python para realizar operaciones en tus datos.

→ **Mapeo objeto-relacional:**

- *Representación de datos como objetos:* Los modelos de Django representan tablas de la base de datos como clases de Python, y los registros como instancias de esas clases. Esto facilita la comprensión y el manejo de los datos.
- *Relaciones entre objetos:* El ORM permite definir relaciones entre diferentes modelos (uno a uno, muchos a uno, muchos a muchos), lo que refleja la estructura de tu base de datos de forma natural.

→ **Generación automática de SQL:**

- *Eliminación de errores comunes:* El ORM se encarga de generar el código SQL necesario para realizar las operaciones de base de datos, lo que reduce significativamente el riesgo de errores sintácticos o de seguridad.
- *Optimización de consultas:* Django ORM puede optimizar las consultas SQL generadas, mejorando el rendimiento de tu aplicación.

ORM - Object Relational Mapper o Mapeador Objeto-Relacional

→ Validación de datos:

- *Integridad de los datos:* Puedes definir reglas de validación en tus modelos para asegurar que los datos ingresados sean correctos y consistentes.
- *Prevención de errores:* La validación se realiza automáticamente al guardar un objeto, lo que ayuda a evitar errores en la aplicación.

→ Facilidad de uso:

- *Sintaxis intuitiva:* La sintaxis del ORM es muy similar a la de Python, lo que facilita su aprendizaje y uso.
- *Gran cantidad de funcionalidades:* El ORM incluye una amplia gama de funcionalidades para filtrar, ordenar, agregar y realizar otras operaciones sobre los datos.

→ Integración con el resto de Django:

- *Flujo de trabajo completo:* El ORM se integra perfectamente con otras partes del framework, como los formularios, las vistas y los templates, lo que facilita el desarrollo de aplicaciones web completas.

Como ves, el ORM de Django nos brinda muchas ventajas con respecto a tener que trabajar con una base de datos por fuera del framework, sin embargo, también podemos mencionar algunas desventajas, como la curva de aprendizaje que, si bien la sintaxis del ORM es intuitiva, puede requerir tiempo aprender a utilizar todas sus funcionalidades. Si las consultas son muy complejas o hay grandes volúmenes de datos, esto puede llegar a afectar el rendimiento también, por lo que para esos casos, conviene hacer consultas directamente por SQL. Sin embargo, para nuestro proyecto final, esta herramienta es la ideal.

ORM - Ejemplo

Usando ORM - Django

- Para ilustrar mejor las ventajas del ORM, y, siguiendo con el ejemplo de la clase anterior veamos un ejemplo sencillo de consultas:
- Vamos a tomar en cuenta los modelos *Autor* y *Libro* de la clase anterior (con algunas variaciones para hacerlo más sencillo):

```
1 from django.db import models
2
3 class Autor(models.Model):
4     nombre = models.CharField(max_length=100)
5     apellido = models.CharField(max_length=100)
6
7 class Libro(models.Model):
8     titulo = models.CharField(max_length=200)
9     autor = models.ForeignKey(Autor, on_delete=models.CASCADE)
10    fecha_publicacion = models.DateField()
```

Utilizando el ORM, podemos realizar consultas como:

- *Obtener todos los libros publicados después del 2020:*

```
12 libros_recientes = Libro.objects.filter(fecha_publicacion__gt='2020-01-01')
```

- *Obtener el número total de libros de un autor específico:*

```
14 autor = Autor.objects.get(id=1)
15 numero_libros = autor.libro_set.count()
```

QuerySets: La herramienta esencial para consultar datos

QuerySets

Lo que acabamos de ver, vamos a utilizarlo mucho en el proyecto final, que es algo que vimos en SQL. Estas consultas o QuerySets son la columna vertebral del ORM de Django, ya que, como vimos en el ejemplo anterior, vamos a poder filtrar, ordenar y realizar otras operaciones (como en SQL, pero ahora desde Django).

Métodos avanzados de los QuerySets:

Filtrado:

- Filtros compuestos: Combinar múltiples filtros usando `&` (AND) y `|` (OR).
- Negación de filtros: Utilizar `~` para negar un filtro.

- Rangos: Usar `__gt`, `__gte`, `__lt`, `__lte` para comparar valores numéricos o fechas.
- Contiene: Utilizar `__contains` para buscar subcadenas dentro de un campo de texto.
- Comienza con/termina con: `__startswith`, `__endswith`.
- Exacto: `__exact` (el valor predeterminado).
- Inexacto: `__iexact` (ignora mayúsculas y minúsculas).

Ordenamiento:

- `order_by`: Ordenar los resultados por uno o más campos, en orden ascendente o descendente.

Agrupación:

- `values`: Agrupar los resultados por uno o más campos y obtener un diccionario para cada grupo.

QuerySets: La herramienta esencial para consultar datos

- **annotate**: Agregar campos calculados a cada grupo.

Otras operaciones:

- **distinct**: Eliminar duplicados.
- **exclude**: Excluir objetos que cumplan cierta condición.
- **union**: Combinar los resultados de dos QuerySets.

Si bien, no vamos a utilizar todas estas operaciones para nuestro proyecto final, te las dejamos acá para darte un campo más general del uso de las QuerySets.

Sin embargo, te volvemos a mencionar que el ORM es muy extenso pero desde acá te brindamos las bases.

Ahora que tenemos un panorama más amplio de las querysets, vamos a abordar las relaciones entre modelos.

Con esto vas a recordar varias cosas de SQL, sobre todo cuando hablábamos de MER.

2

Relaciones entre modelos

Algo fundamental, como ya lo vimos en SQL, son las relaciones, y en este caso, en Django, veremos que se soportan 3 tipos de relaciones. ¡Vamos a verlas!



Relaciones entre modelos con Django

1:1, 1:N, N:M

Las relaciones son fundamentales para modelar la estructura de tus datos. Para estos ejemplos, veremos tres tipos principales de relaciones:

- **Uno a uno:** Una instancia de un modelo está relacionada con una y solo una instancia de otro modelo.
- **Uno a muchos:** Muchas instancias de un modelo están relacionadas con una sola instancia de otro modelo.
- **Muchos a muchos:** Muchas instancias de un modelo están relacionadas con muchas instancias de otro modelo.

Lo vemos con ejemplos: N:1

Para no perder el hilo, vamos a seguir con el ejemplo de *Libro* y *Autor* (un poco más simplificado):

```

1  from django.db import models
2
3  class Autor(models.Model):
4      nombre = models.CharField(max_length=100)
5
6  class Libro(models.Model):
7      titulo = models.CharField(max_length=200)
8      autor = models.ForeignKey(Autor, on_delete=models.CASCADE)

```

En este ejemplo, un autor puede escribir muchos libros, pero un libro solo puede tener un autor - relación **muchos a uno** -.

Relaciones entre modelos con Django

Lo vemos con ejemplos: 1:1

Ahora cambiamos un poco el ejemplo y vamos a algo más concreto que te puede servir para el proyecto final 😊

En este ejemplo, tenemos un modelo *Persona* y un modelo *Perfil*. Cada persona tiene un único perfil, y cada perfil pertenece a una única persona.

OneToOneField establece la relación **uno a uno** entre *Persona* y *Perfil*. Si eliminamos una persona, su perfil asociado también se eliminará debido a ***on_delete=models.CASCADE***.

```
1  from django.db import models
2
3  class Persona(models.Model):
4      nombre = models.CharField(max_length=100)
5      apellido = models.CharField(max_length=100)
6
7
8  class Perfil(models.Model):
9      persona = models.OneToOneField(Persona, on_delete=models.CASCADE)
10     biografia = models.TextField()
11     fecha_nacimiento = models.DateField()
```

Relaciones entre modelos con Django

Lo vemos con ejemplos: N:M

Por último, cuando una instancia de un modelo esté relacionada con muchas instancias de otro modelo y viceversa.

```

1  from django.db import models
2
3  class Estudiante(models.Model):
4      nombre = models.CharField(max_length=100)
5
6  class Curso(models.Model):
7      nombre = models.CharField(max_length=100)
8      estudiantes = models.ManyToManyField(Estudiante)
```

Para este caso tenemos como ejemplo el modelo *Estudiante* y el modelo *Curso*. Un estudiante puede tomar muchos cursos, y un curso puede tener muchos estudiantes inscriptos.

ManyToManyField establece la relación **muchos a muchos** entre *Estudiante* y *Curso*. Django crea automáticamente una tabla intermedia para gestionar esta relación.

Consideraciones adicionales:

- **on_delete**: Este argumento en *ForeignKey* y *OneToOneField* especifica qué sucede con el objeto relacionado cuando se elimina el objeto padre. **CASCADE** elimina el objeto relacionado, **PROTECT** impide la eliminación si existen objetos relacionados, **SET_NULL** establece el campo relacionado a null, etc.
- **Tabla intermedia**: En las relaciones muchos a muchos, Django crea una tabla intermedia para almacenar las asociaciones entre los dos modelos. Se pueden personalizar esta tabla si es necesario.
- **Relaciones inversas**: Cada relación tiene una relación inversa. Por ejemplo, en la relación *Estudiante* y *Curso*, se puede acceder a los cursos de un estudiante a través de `estudiante.curso_set.all()`.

3

Operaciones CRUD con el ORM de Django

Las operaciones CRUD (Create, Read, Update, Delete) son las cuatro operaciones básicas que se pueden realizar en una base de datos para gestionar los datos, así como lo vimos en SQL.

Ahora vamos a verlas utilizando el ORM de Django.

CRUD: Create

Create - Crear

→ Create:

La operación *Create* se utiliza para **insertar** nuevos registros en la base de datos. Esto se hace creando una instancia de un modelo (que representa una tabla) y luego agregándolo.

Ejemplo:

Vamos a volver al ejemplo del modelo persona, donde para crear un nuevo registro en la base de datos, lo hacemos de la siguiente manera →

```
persona = Persona(nombre='Infor', apellido='Matorio')
persona.save()
```

Create - Múltiples instancias

```
15 personas = [
16     Persona(nombre='Evelyn', apellido='Guerra'),
17     Persona(nombre='Cris', apellido='Morales'),
18 ]
19 Persona.objects.bulk_create(personas)
```

Con el comando **bulk_create** vamos a hacer más eficiente la creación de múltiples objetos a la vez.

CRUD: Read

Read - Leer

→ Obtener todos los objetos:

Como ya vimos en un ejemplo anterior, con `clase.object.all()` podemos obtener todos los registros de nuestro modelo `Persona`.

```
todas_las_personas = Persona.objects.all()
```

→ Filtrar objetos:

Además podemos filtrar, como en el siguiente ejemplo, para obtener el registro de un apellido buscado.

```
personas_apellido_perez = Persona.objects.filter(apellido='Morales')
```

→ Obtener un objeto específico:

```
persona = Persona.objects.get(id=1) # Cuando buscamos por ID
```

Para este caso, **get** lanza una excepción si no encuentra el objeto, así que, si no estamos seguros que el registro existe, lo recomendable es usar **filter**.

CRUD: Update

Update - actualizar/modificar

→ Update:

La operación *Update* se utiliza para modificar registros existentes en la base de datos. Esto se hace recuperando el objeto que deseamos modificar, realizando los cambios y luego confirmando los cambios.

```
persona.nombre = 'Juan Carlos'  
persona.save()
```

Update - Para múltiples instancias

```
Persona.objects.filter(apellido='Morales').update(apellido='Moral')
```

Como vés, buscamos y actualizamos en este ejemplo.

CRUD: Delete

Delete- eliminar

→ Delete:

La operación Delete se utiliza para eliminar registros de la base de datos. Esto se hace recuperando el objeto que deseamos eliminar y luego eliminándolo.

```
persona.delete()
```

Delete - Para múltiples instancias

```
Persona.objects.filter(apellido='Moral').delete()
```

Si te diste cuenta, podemos combinar varios comandos para utilizar las operaciones CRUD.

Todos estos ejemplos, te van a ser útiles para el proyecto final, por lo que te recomendamos que a medida que avancemos con el proyecto, puedas ir practicando con lo que te brindamos en esta clase.



Lo que vimos hoy

Como te recomendamos recién, a medida que más cosas vayamos viendo en el proyecto final, más vas a poder practicar con las herramientas que te vamos dando. Entre el material complementario y estas clases, tratamos de brindarte bases para que puedas desenvolverte con Django.

¡No dejes de practicar!





**¡Nos vemos
En la próxima
clase!**

